ArcLoom LoomCore Specification for RNA 3D Folding

Kaggle-to-Real-World Bridge (Two-Part Spec)

Version: v0.1 Status: Draft (validation-first)
Date: January 13, 2026

0. Document control

Purpose. This document preserves a concrete, testable, end-to-end specification for a *compute-minimizing* RNA 3D inference pipeline (**LoomCore**) that was developed in the ArcLoom discussions. The specification is intentionally split into:

- **Part 1 (Kaggle score):** a reproducible, deterministic pipeline that produces a valid competition submission and yields an empirical metric (leaderboard score) with minimal compute.
- **Part 2 (Real-world):** the same LoomCore pipeline hardened for deployment outside Kaggle (traceability, versioning, validation, clinical-grade documentation patterns).

What this spec is not. This is not a claim of a new physics substrate. It is a buildable architecture and workflow that: (a) uses structural priors (templates), (b) uses deterministic geometry (Kabsch), and (c) uses an explicit, auditable process flow to reduce brute-force compute.

Conformance language. "Shall" indicates a requirement. "Should" indicates a recommendation. "May" indicates an option.

1. Scope and intended use

1.1 Intended use

The LoomCore system is intended to:

- Ingest RNA sequence(s) and optional template structures.
- Produce a 3D coordinate prediction per residue for each target.
- Provide auditable logs of: which templates were used, alignment mappings, injection decisions, and output normalization.

1.2 Non-intended use

This specification does not define a clinical diagnostic device. Any clinical use would require an additional product safety file, hazard analysis, and regulated validation plan.

2. High-level architecture

2.1 System decomposition

LoomCore is decomposed into five deterministic blocks:

1. **I/O and dataset normalization**
2. **Template search (coarse filter)**
3. **Alignment & mapping (fine filter)**
4. **PFSC fill (physics-field structural completion)**
5. **Template injection & output controls**

Each block shall be testable in isolation (unit tests) and in sequence (integration tests).

2.2 Key design principle

The pipeline shall reduce compute by narrowing the hypothesis space early, using structure. Concretely:

- Use k-mer overlap to eliminate most templates quickly.
- Use a deterministic sequence alignment to map indices.
- Use geometry-only fitting (Kabsch) instead of iterative deep model alignment.
- Use a PFSC completion engine to fill missing coordinates where templates do not cover.

3. Terms, definitions, abbreviations

Term	Definition
LoomCore	The deterministic RNA 3D inference pipeline defined in this document (templates + alignment + PFSC + injection + controls).
PFSC	Physics Field Structural Completion: a deterministic completion engine that infers missing structure from partial constraints.
Template	A known RNA structure used as a prior (coordinate donor) for a target sequence via alignment.
Injection	Overriding some predicted coordinates with template-provided coordinates after aligning coordinate frames.
Kabsch fit	Closed-form least-squares rigid alignment (rotation + translation) between two point sets.
Balanced ternary	A three-symbol arithmetic system with digits in $\{-1, 0, +1\}$; used conceptually for ArcLoom, not required for Part 1.
GDP/GMP	Good Documentation Practice / Good Manufacturing Practice (quality system patterns; used as guidance in Part 2).

4. Part 1 — Kaggle score specification (normative)

4.1 Objective

Part 1 shall produce a Kaggle-valid submission file (CSV) and a run log that allows reproduction of the submission from the same inputs.

4.2 Inputs

The Part 1 pipeline shall accept:

- A Kaggle competition input dataset containing target sequences and required IDs.
- A template library dataset containing sequence records and coordinates.
- A PFSC engine package (software dependency) that can accept templates and emit 3D coordinates.

4.3 Outputs

Part 1 shall emit:

- `submission.csv` formatted to Kaggle rules.
- `template_capture/` folder containing per-target template metadata and mappings.
- `runlog.txt` capturing version, config, random seed(s), and timing.

4.4 Process flow (Input → Process → Output)

4.4.1 Block A: Dataset normalization

Input: target IDs and sequences; template sequences and coordinates.

Process:

- Validate presence of required columns and file paths.
- Normalize sequences to uppercase {A,C,G,U} (others shall be mapped to `N` and excluded from k-mers).

Output: normalized target list; normalized template index.

4.4.2 Block B: Template search (k-mer overlap)

Input: target sequence s; template library sequences $\{t_i\}$.

Process:

1. Choose k-mer length k (default $k = 5$).

2. Compute the target k-mer set:

$$K(s) = \{s[j : j + k] \mid 0 \le j \le |s| - k,\ s[j : j + k] \subset \{A, C, G, U\}\}.$$

3. For each template t_i, compute $K(t_i)$ and overlap score:

$$\text{overlap}(s, t_i) = \frac{|K(s) \cap K(t_i)|}{\max(1, |K(s)|)}.$$

4. Rank templates by overlap and keep the top M candidates (default $M = 200$).

Output: candidate template list $\mathcal{C}(s)$.

4.4.3 Block C: Alignment & index mapping

Input: target sequence s and candidate templates $\mathcal{C}(s)$.

Process:

- Perform a deterministic global alignment between s and t (Needleman–Wunsch) with scoring:

$$\text{match} = +2, \quad \text{mismatch} = -1, \quad \text{gap} = -2.$$

- Produce an index mapping $\pi_{s \to t}$ from target positions to template positions.
- Compute coverage:

$$\text{cov} = \frac{\#\{j \mid \pi_{s \to t}(j) \text{ exists and template coord finite}\}}{|s|}.$$

- Compute identity over mapped positions:

$$\text{ident} = \frac{\#\{j \mid \pi_{s \to t}(j) \text{ exists and } s[j] = t[\pi(j)]\}}{\max(1, \#\{j \mid \pi(j) \text{ exists}\})}.$$

- Compute a composite template quality score (default weights):

$$Q = \text{cov} \cdot (0.7 \cdot \text{ident} + 0.3 \cdot \text{overlap}).$$

- Keep the top N templates (default $N = 3$) by Q.

Output: a ranked template set $\mathcal{T}(s)$ with mappings $\pi_{s \to t}$.

4.4.4 Block D: PFSC fill

Input: target s and template constraints from $\mathcal{T}(s)$.

Process:

- Convert template coordinates into target-indexed constraints using $\pi_{s \to t}$.
- Run PFSC completion to infer missing 3D coordinates.
- PFSC shall be deterministic given the same inputs and seed (if any).

Output: a full coordinate set $\hat{X}(s) \in \mathbb{R}^{|s| \times 3}$.

4.4.5 Block E: Template injection & output controls

Input: PFSC coordinates $\hat{X}(s)$ and template coordinates X_t for $t \in \mathcal{T}(s)$.

Process:

1. For each template t, compute an optional rigid alignment (Kabsch) between the subset of points where both $\hat{X}$ and X_t are finite:

$$
\begin{aligned}
&P = \{\hat{X}_j\},\ Q = \{(X_t)_{\pi(j)}\} \\
&\bar{P} = \frac{1}{m}\sum P, \quad \bar{Q} = \frac{1}{m}\sum Q \\
&H = (P - \bar{P})^\top (Q - \bar{Q}) \\
&H = U\Sigma V^\top, \quad R = VU^\top, \quad \det(R) \geq 0 \text{ (enforce)} \\
&t = \bar{Q} - R\bar{P}.
\end{aligned}
$$

2. Apply R, t to transform PFSC coordinates into template frame (if $m \geq 5$ overlap points).
3. Inject template coordinates into transformed PFSC coordinates at positions where template is finite and alignment confidence is acceptable.
4. Output control: if any coordinate component violates a clipping range (e.g., < -999.999), apply a translation shift to bring the minimum inside range without changing relative geometry:

$$X' = X + \Delta, \quad \Delta = (L - \min(X)) + \epsilon,\ \text{component-wise.}$$

Output: final coordinates $X_{\text{out}}(s)$ and a per-target injection log.

4.5 Determinism requirements

Part 1 shall be deterministic under:

- Fixed dataset versions (Kaggle inputs).
- Fixed template selection config (k, M, N, weights, scoring).
- Fixed PFSC version and any internal random seeds.

The run log shall record all parameters and dataset checksums.

4.6 Acceptance criteria (Part 1)

A Part 1 run shall be considered **PASS** if:

- A Kaggle submission file is produced and passes Kaggle format validation.
- Re-running the pipeline with the same inputs yields byte-identical `submission.csv`.

4.7 Operator procedure (Kaggle notebook, no dev setup)

This is the recommended execution path for Part 1.

Inputs: Kaggle notebook with the competition dataset and the PFSC dataset added as "Inputs".

Process:

1. Create a new Kaggle notebook in the competition.
2. In the notebook "Data" tab, add:
   - the competition dataset (required),
   - the PFSC dataset referenced by the LoomCore script (required).
3. Create one code cell and paste the contents of `loomcore_kaggle_baseline.py`.
4. Run the notebook.

Outputs:

- `submission.csv` written to the Kaggle working directory,
- `template_capture/` written to the working directory.

Acceptance check: Submit `submission.csv` to Kaggle and record the score in the run manifest.

4.8 Operator procedure (local run, optional)

If you want to run outside Kaggle, the recommended option is a Docker image (to pin dependencies). This spec does not provide the Dockerfile; instead it requires that a Part 2 release provide a single-command local runner.

Recommended next step: After Kaggle reproducibility is confirmed, create a minimal local runner that accepts:

- a folder of targets,
- a folder of templates,
- an output folder,
- a pinned config YAML.

5. Part 2 — Real-world specification (normative with GDP/GMP patterns)

5.1 Objective

Part 2 shall produce a deployable LoomCore inference service that:

- Accepts sequences via a stable API (CLI or HTTP).

- Produces structures plus a quality report (confidence, template provenance).
- Emits an immutable audit trail (inputs, parameters, outputs, hashes).

5.2 Deployment options

Option RW-S (recommended): Software-only deployment on commodity CPU/GPU.

Option RW-H (future): Optional FPGA acceleration for the heaviest deterministic kernels (k-mer overlap, alignment DP, Kabsch).

This document specifies RW-S as the baseline. RW-H is informative.

5.3 Inputs

- One or more RNA sequences (FASTA or JSON).
- Optional user-provided template coordinates (PDB/mmCIF) with explicit provenance.
- A pinned, versioned template library (curated).

5.4 Outputs

- Predicted structure(s): PDB/mmCIF + machine-readable JSON.
- A "provenance bundle": templates used, mappings, injection sites, Kabsch residuals.
- A "run manifest": code version, parameters, environment fingerprint, hashes.

5.5 Quality system controls (GDP/GMP style)

For Part 2, the project shall maintain:

- **Requirements traceability:** each requirement in this spec maps to a test case ID.
- **Change control:** changes require version bumps and a changelog.
- **Verification:** unit + integration tests with stored golden outputs.
- **Validation:** evaluation on a non-Kaggle reference set (e.g., PDB-derived RNA structures) with predefined metrics.

5.6 Real-world acceptance criteria (Part 2)

A Part 2 release shall be considered **PASS** if:

- It reproduces Part 1 results when run on the Kaggle dataset (sanity cross-check).
- It produces stable outputs for a fixed reference dataset with no regressions across versions (within tolerance).
- It produces a complete provenance bundle for each inference.

6. Decision log (approved baseline)

This section records high-impact architectural decisions and their rationale.

ID	Decision	Rationale
D-01	Coarse-to-fine template search	Reduces compute by pruning templates early using k-mer overlap.
D-02	Deterministic global alignment	Ensures reproducible index mapping and auditability.
D-03	Kabsch frame alignment for injection	Closed-form, fast, deterministic geometry fit (no iterative optimizer).
D-04	Output clipping control via translation	Guarantees downstream format constraints without changing internal geometry.
D-05	Two-part roadmap (Kaggle → real world)	Forces an empirical metric first, then hardens for deployment.

7. Engineering headwinds (what can break, and how we test it)

This is the "headwinds list" that must be tackled one-by-one for a credible real-world system:

1. **Template bias / leakage:** templates may overfit competition distribution.
   Mitigation: separate curated template sets; evaluate on external benchmarks.
2. **Alignment failures:** long indels or repeats can corrupt $\pi_{s\to t}$.
   Mitigation: sanity checks on mapping monotonicity; fallback to local alignment windows.
3. **Injection overconfidence:** overwriting PFSC with wrong template sections.
   Mitigation: inject only where Kabsch residual is below threshold; keep multiple hypotheses.
4. **PFSC drift across versions:** different PFSC versions change outputs.
   Mitigation: pin versions; record hashes; golden tests.
5. **Compute budget:** Kaggle runtime limits and real-world throughput.
   Mitigation: cache k-mer sets and alignments; batch inference; optional RW-H acceleration.
6. **Traceability burden:** GDP/GMP adds paperwork and process overhead.
   Mitigation: automate manifest generation; enforce run manifests as artifacts.

8. Recommended next steps

Recommended next step (do this first): Lock Part 1 reproducibility.

1. Run LoomCore Part 1 twice and confirm byte-identical `submission.csv`.
2. Record the Kaggle public score and store it in the run manifest.
3. Create a tiny "reference pack" (10 targets) and freeze its outputs as golden files.

Then: Build Part 2 scaffolding (manifests, provenance bundles, test harness), before chasing performance.

Appendix A — Baseline Kaggle reference implementation (informative)

The user provided a baseline run script implementing a LoomCore-like pipeline (templates + PFSC + injection). It should be treated as the reference implementation for Part 1 until replaced by a version-controlled repository.

This appendix includes the baseline script as provided (copied to `loomcore_kaggle_baseline.py` for stable filenames).

A.1 Script (verbatim)

```
# DSF PFSC submission + multi-template ensemble injection (v4)
# Goal:
# - Find top N templates per target using pdb_seqres_NA.fasta + PDB_RNA/*.cif
# - Fill models 1..N with those templates (hybrid: template coords + PFSC fill aligned via Kabsch)
# - Leave model 5 as "pure PFSC" (optionally: run PFSC again with use_templates=False)
#
# How to use in Kaggle:
# 1) Add inputs: stanford-rna-3d-folding-2, dsf-ai-rna3d-app-pfsc
# 2) Put this entire file into a cell with %%writefile run.py, then run: !python run.py
#
# Notes:
# - Best-of-5 scoring means bad extra models don't hurt; diversity helps.
# - We rigid-align PFSC coords into each template frame before blending, to avoid frame mismatch.

import os, sys, time, re, inspect
from pathlib import Path
from datetime import datetime, date
import numpy as np
import pandas as pd

# ------------------------------
# User-tunable knobs
# ------------------------------
N_TEMPLATES = 4 # models 1..4 will be template-hybrids
ALIGN_TOPK = 40 # how many candidate templates to fully align per chain (speed/quality tradeoff)
LEN_WINDOW_FRAC = 0.35 # seqres length window around target length
LEN_WINDOW_MIN = 50
MIN_COVERAGE_KEEP = 0.15 # discard templates with too little coordinate coverage

```

```
PURE_PFSC_MODEL5 = True # if True, run a second PFSC pass with use_templates=False and use that for model_5
PURE_PFSC_SEED = 777

# ------------------------------
# Paths (Kaggle mount points)
# ------------------------------
COMP_DIR = Path("/kaggle/input/stanford-rna-3d-folding-2")
DSF_DIR = Path("/kaggle/input/dsf-ai-rna3d-app-pfsc")

# Find DSF code folder
candidate = DSF_DIR / "dsf_ai_rna3d_app_PFSC"
if (candidate / "rna3d_app" / "predict.py").exists():
    DSF_CODE_DIR = candidate
else:
    DSF_CODE_DIR = None
    for sub in DSF_DIR.iterdir():
        if sub.is_dir() and (sub / "rna3d_app" / "predict.py").exists():
            DSF_CODE_DIR = sub
            break
    if DSF_CODE_DIR is None:
        raise FileNotFoundError("Could not find rna3d_app/predict.py inside DSF dataset input.")

sys.path.insert(0, str(DSF_CODE_DIR))
from rna3d_app import predict as dsf_predict
from rna3d_app.l5_physics import template_guided_init as tgi

print("DSF code dir:", DSF_CODE_DIR)
print("predict.py :", dsf_predict.__file__)
print("predict_submission signature:", inspect.signature(dsf_predict.predict_submission))

# ------------------------------
# Fix DSF's template imports (varies by DSF build)
# ------------------------------
if hasattr(tgi, "build_template_init_coords"):
    dsf_predict.build_template_init_coords = tgi.build_template_init_coords

if hasattr(tgi, "load_release_dates_map"):
    dsf_predict.load_release_dates_map = tgi.load_release_dates_map
elif hasattr(tgi, "load_release_dates"):
    dsf_predict.load_release_dates_map = tgi.load_release_dates
else:
    raise AttributeError("template_guided_init has no load_release_dates_map or load_release_dates")

# ------------------------------
# Dataset check
# ------------------------------
PDB_DIR = COMP_DIR / "PDB_RNA"
SEQRES_PATH = PDB_DIR / "pdb_seqres_NA.fasta"
RELEASE_PATH = PDB_DIR / "pdb_release_dates_NA.csv"

```

```
print("\nDATASET CHECK")
print("COMP_DIR:", COMP_DIR)
print("PDB_DIR :", PDB_DIR, "exists=", PDB_DIR.exists())
print("SEQRES :", SEQRES_PATH, "exists=", SEQRES_PATH.exists())
print("RELEASE :", RELEASE_PATH, "exists=", RELEASE_PATH.exists())
if PDB_DIR.exists():
    n_cif = len(list(PDB_DIR.glob("*.cif")))
    n_gz = len(list(PDB_DIR.glob("*.cif.gz")))
    print(f"PDB_RNA files: {n_cif} x .cif | {n_gz} x .cif.gz")

# ------------------------------
# Helpers: seqres parsing, CIF loading, alignment, Kabsch
# ------------------------------
_PDBCHAIN_RE = re.compile(r"([0-9A-Za-z]{4})[_:|]([A-Za-z0-9])")
_SEQRES_CACHE = {}
_CIF_CACHE = {}

def _parse_date(x):
    if isinstance(x, date):
        return x
    try:
        return datetime.strptime(str(x)[:10], "%Y-%m-%d").date()
    except Exception:
        return None

def _parse_pdb_chain_from_header(h: str):
    m = _PDBCHAIN_RE.search(h)
    if m:
        return m.group(1).lower(), m.group(2)
    tok = h.strip().split()[0].lstrip(">")
    parts = re.split(r"[|_:]", tok)
    pdb = None
    chain = None
    for i, p in enumerate(parts):
        if len(p) == 4:
            pdb = p.lower()
            if i + 1 < len(parts) and len(parts[i+1]) == 1:
                chain = parts[i+1]
            break
    return pdb, chain

def _kmer_set(seq: str, k: int = 5):
    mapping = {'A':0, 'C':1, 'G':2, 'U':3, 'T':3}
    mask = (1 << (2*k)) - 1
    code = 0
    filled = 0
    out = set()
    for ch in seq:
        v = mapping.get(ch.upper(), None)
        if v is None:
            code = 0
            filled = 0
            continue
        code = ((code << 2) | v) & mask
```

```
        filled = filled + 1 if filled < k else k
        if filled == k:
            out.add(code)
    return out

def _load_seqres_db(seqres_path: Path):
    key = str(seqres_path)
    if key in _SEQRES_CACHE:
        return _SEQRES_CACHE[key]

    records = []
    header = None
    buf = []

    def flush(h, s):
        if not h:
            return
        pdb, ch = _parse_pdb_chain_from_header(h)
        if not pdb or not ch:
            return
        seq = s.replace(" ", "").replace("\n", "").strip()
        if not seq:
            return
        km = _kmer_set(seq, k=5)
        records.append((len(seq), pdb, ch, seq, km))

    with open(seqres_path, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if line.startswith(">"):
                if header is not None:
                    flush(header, "".join(buf))
                header = line[1:]
                buf = []
            else:
                buf.append(line)
        if header is not None:
            flush(header, "".join(buf))

    records.sort(key=lambda x: x[0])
    _SEQRES_CACHE[key] = records
    print(f"Loaded seqres DB: {len(records)} chains from {seqres_path}")
    return records

def _get_cif_coords(pdb_dir: Path, pdb_id: str):
    pdb_id = pdb_id.lower()
    if pdb_id in _CIF_CACHE:
        return _CIF_CACHE[pdb_id]

    cif_path = pdb_dir / f"{pdb_id}.cif"
    if not cif_path.exists():
        cif_path = pdb_dir / f"{pdb_id.upper()}.cif"
```

```
    if not cif_path.exists():
        _CIF_CACHE[pdb_id] = None
        return None

    fn = getattr(tgi, "load_c1_coords_from_cif", None)
    if fn is None:
        _CIF_CACHE[pdb_id] = None
        return None

    try:
        coords = fn(cif_path)
    except TypeError:
        coords = fn(pdb_dir, pdb_id)

    _CIF_CACHE[pdb_id] = coords
    return coords

def _global_align_map(template_seq: str, target_seq: str, match=2, mismatch=-1, gap=-2):
    n = len(template_seq)
    m = len(target_seq)
    dp = np.zeros((n+1, m+1), dtype=np.int32)
    tb = np.zeros((n+1, m+1), dtype=np.uint8) # 0 diag, 1 up, 2 left

    for i in range(1, n+1):
        dp[i,0] = dp[i-1,0] + gap
        tb[i,0] = 1
    for j in range(1, m+1):
        dp[0,j] = dp[0,j-1] + gap
        tb[0,j] = 2

    for i in range(1, n+1):
        ai = template_seq[i-1]
        for j in range(1, m+1):
            bj = target_seq[j-1]
            s = match if ai == bj else mismatch
            diag = dp[i-1, j-1] + s
            up = dp[i-1, j] + gap
            left = dp[i, j-1] + gap
            if diag >= up and diag >= left:
                dp[i,j] = diag; tb[i,j] = 0
            elif up >= left:
                dp[i,j] = up; tb[i,j] = 1
            else:
                dp[i,j] = left; tb[i,j] = 2

    mapping = [-1] * m
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and tb[i,j] == 0:
            mapping[j-1] = i-1
            i -= 1; j -= 1
        elif i > 0 and (j == 0 or tb[i,j] == 1):
            i -= 1
```

```
        else:
            j -= 1
    return mapping

def _kabsch_align(P: np.ndarray, Q: np.ndarray):
    """
    Find rotation R and translation t minimizing || (P @ R + t) - Q || over rows.
    P,Q: (n,3)
    """
    P = P.astype(np.float64, copy=False)
    Q = Q.astype(np.float64, copy=False)
    Pc = P - P.mean(axis=0, keepdims=True)
    Qc = Q - Q.mean(axis=0, keepdims=True)
    C = Pc.T @ Qc
    V, S, Wt = np.linalg.svd(C)
    d = np.sign(np.linalg.det(V @ Wt))
    D = np.diag([1.0, 1.0, d])
    R = V @ D @ Wt
    t = Q.mean(axis=0) - (P.mean(axis=0) @ R)
    return R, t

# ------------------------------
# Load release dates
# ------------------------------
def _load_release_dates_map_safe(dataset_root: Path):
    try:
        rd = dsf_predict.load_release_dates_map(dataset_root)
    except Exception:
        try:
            rd = dsf_predict.load_release_dates_map(str(dataset_root))
        except Exception:
            rd = {}
    if rd is None:
        rd = {}
    # normalize keys to lowercase
    if isinstance(rd, dict):
        out = {}
        for k, v in rd.items():
            if not k:
                continue
            out[str(k).lower()] = v
        return out
    return {}

RELEASE_DATES = _load_release_dates_map_safe(COMP_DIR)
print("Loaded release_dates entries:", len(RELEASE_DATES))

# ------------------------------
# Multi-template search per chain
# ------------------------------
SEQRES_RECORDS = _load_seqres_db(SEQRES_PATH)
SEQRES_LENS = [r[0] for r in SEQRES_RECORDS]

import bisect
```

```
295
296 def _allowed_by_date(pdb_id: str, cutoff: date):
297     d = RELEASE_DATES.get(pdb_id.lower(), None)
298     # If unknown date, allow; otherwise enforce cutoff.
299     return (d is None) or (d <= cutoff)
300
301 def _find_chain_templates(target_seq: str, pdb_dir: Path, cutoff: date, n_templates:
        int):
302     L = len(target_seq)
303     tkm = _kmer_set(target_seq, k=5)
304     if not tkm:
305         return []
306
307     win = max(LEN_WINDOW_MIN, int(LEN_WINDOW_FRAC * L))
308     lo = bisect.bisect_left(SEQRES_LENS, max(1, L - win))
309     hi = bisect.bisect_right(SEQRES_LENS, L + win)
310
311     # Rough rank by k-mer overlap
312     rough = []
313     denom = max(1, len(tkm))
314     for (lseq, pdb, ch, seq, km) in SEQRES_RECORDS[lo:hi]:
315         if not _allowed_by_date(pdb, cutoff):
316             continue
317         inter = len(tkm & km)
318         if inter == 0:
319             continue
320         score = inter / denom
321         rough.append((score, pdb, ch, seq))
322
323     if not rough:
324         return []
325
326     rough.sort(reverse=True, key=lambda x: x[0])
327     rough = rough[:max(ALIGN_TOPK, n_templates)] # cap compute
328
329     cand = []
330     seen = set()
331
332     for kmer_score, pdb, ch, seq in rough:
333         key = (pdb, ch)
334         if key in seen:
335             continue
336         seen.add(key)
337
338         cif = _get_cif_coords(pdb_dir, pdb)
339         if not cif:
340             continue
341         c1 = cif.get(ch)
342         if not c1:
343             continue
344
345         m = _global_align_map(seq, target_seq)
346
347         xyz = np.full((L, 3), np.nan, dtype=np.float32)
```

```
        mapped = 0
        matches = 0

        for j, ti in enumerate(m):
            if ti < 0:
                continue
            mapped += 1
            if seq[ti] == target_seq[j]:
                matches += 1
            label_seq_id = ti + 1
            if label_seq_id in c1:
                xyz[j] = c1[label_seq_id].astype(np.float32)

        finite = np.isfinite(xyz[:, 0])
        cov = float(finite.sum()) / float(max(1, L))
        if cov < MIN_COVERAGE_KEEP:
            continue

        ident = float(matches) / float(max(1, mapped))
        # blended quality metric: identity + rough kmer, scaled by coverage
        qual = cov * (0.7 * ident + 0.3 * float(kmer_score))

        cand.append({
            "pdb": pdb,
            "chain": ch,
            "kmer": float(kmer_score),
            "ident": ident,
            "cov": cov,
            "qual": qual,
            "xyz": xyz,
            "seq": seq,
        })

        # Early stop if we already have plenty of high-quality options
        if len(cand) >= max(n_templates, 8) and max(c["qual"] for c in cand) > 0.95:
            # don't break too early; still allow some diversity
            pass

    if not cand:
        return []

    cand.sort(key=lambda d: d["qual"], reverse=True)
    return cand[:n_templates]

# ------------------------------
# Capture templates per target (for later injection)
# ------------------------------
TEMPLATE_CAPTURE = {} # tid -> {templates: [xyz_global], meta:[str], cov:[float]}

def _chain_key_order(unique_chain_seqs):
    if isinstance(unique_chain_seqs, dict):
        keys = list(unique_chain_seqs.keys())
    else:
        keys = list(range(len(unique_chain_seqs)))
```

```
    # stable numeric-ish ordering
    try:
        keys_sorted = sorted(keys, key=lambda x: int(x) if str(x).isdigit() else
            str(x))
    except Exception:
        keys_sorted = keys
    return keys_sorted

def _concat_global(coords_by_chain, chain_keys):
    parts = []
    for k in chain_keys:
        arr = coords_by_chain.get(k)
        if arr is None:
            continue
        parts.append(arr)
    if not parts:
        return None
    return np.concatenate(parts, axis=0)

# Keep original DSF template init (if it works) to avoid breaking anything
_ORIG_TEMPLATE_INIT = dsf_predict.build_template_init_coords
_ORIG_SIG = None
try:
    _ORIG_SIG = inspect.signature(_ORIG_TEMPLATE_INIT)
except Exception:
    _ORIG_SIG = None

def build_template_init_coords_patched(
    target_id,
    unique_chain_seqs,
    msa_dir,
    pdb_dir,
    temporal_cutoff,
    release_dates=None,
    max_templates=250,
    **kwargs
):
    dataset_root = Path(kwargs.get("dataset_root", Path(msa_dir).parent))
    pdb_dir = Path(pdb_dir)
    if (not pdb_dir.exists()) or (not any(pdb_dir.glob("*.cif*"))):
        if (dataset_root / "PDB_RNA").exists():
            pdb_dir = dataset_root / "PDB_RNA"

    cutoff = _parse_date(temporal_cutoff) or date.today()

    # 1) Try DSF's original MSA-based approach first (nice if it hits)
    try:
        wants_release_dates = False
        if _ORIG_SIG is not None:
            wants_release_dates = ("release_dates" in _ORIG_SIG.parameters) or
                (len(_ORIG_SIG.parameters) >= 6)

        if wants_release_dates:
            coords_by_chain, msg = _ORIG_TEMPLATE_INIT(
```

```
            target_id, unique_chain_seqs, msa_dir, pdb_dir, temporal_cutoff,
                RELEASE_DATES,
            max_templates=max_templates
        )
    else:
        coords_by_chain, msg = _ORIG_TEMPLATE_INIT(
            target_id, unique_chain_seqs, msa_dir, pdb_dir, temporal_cutoff,
            max_templates=max_templates
        )

    if coords_by_chain and any(np.isfinite(v).any() for v in
        coords_by_chain.values()):
        # Still capture multiple templates via seqres for injection diversity
        # but return MSA-based coords to DSF for initialization.
        pass
except Exception:
    coords_by_chain, msg = {}, "MSA failed"

# 2) Always compute seqres templates for capture + DSF init fallback
chain_keys = _chain_key_order(unique_chain_seqs)

if isinstance(unique_chain_seqs, dict):
    chain_items = [(k, unique_chain_seqs[k]) for k in chain_keys]
else:
    chain_items = [(k, unique_chain_seqs[k]) for k in chain_keys]

per_chain_templates = {}
best_coords_by_chain = {}

for k, seq in chain_items:
    cand = _find_chain_templates(seq, pdb_dir=pdb_dir, cutoff=cutoff,
        n_templates=N_TEMPLATES)
    per_chain_templates[k] = cand
    if cand:
        best_coords_by_chain[k] = cand[0]["xyz"]
    else:
        best_coords_by_chain[k] = np.full((len(seq), 3), np.nan, dtype=np.float32)

# Build N global templates (index i uses i-th candidate per chain if available,
    else best)
global_templates = []
meta = []
covs = []

total_len = sum(len(seq) for _, seq in chain_items)

for i in range(N_TEMPLATES):
    coords_i = {}
    meta_parts = []
    for k, seq in chain_items:
        cand = per_chain_templates.get(k, [])
        use = cand[i] if i < len(cand) else (cand[0] if cand else None)
        if use is None:
            coords_i[k] = np.full((len(seq), 3), np.nan, dtype=np.float32)
```

```
                meta_parts.append(f"{k}:none")
            else:
                coords_i[k] = use["xyz"]
                meta_parts.append(f"{k}:{use['pdb']}|{use['chain']}
                    q={use['qual']:.2f} cov={use['cov']:.2f}")
        xyz_global = _concat_global(coords_i, chain_keys)
        if xyz_global is None:
            continue
        cov_global = float(np.isfinite(xyz_global[:,0]).sum()) / float(max(1,
            total_len))
        global_templates.append(xyz_global)
        meta.append("; ".join(meta_parts))
        covs.append(cov_global)

    TEMPLATE_CAPTURE[str(target_id)] = {
        "templates": global_templates,
        "meta": meta,
        "cov": covs,
        "cutoff": str(cutoff),
    }

    # Return best coords to DSF (to anchor PFSC)
    # Prefer MSA-based coords if they worked, else best seqres coords.
    if coords_by_chain and any(np.isfinite(v).any() for v in
        coords_by_chain.values()):
        return coords_by_chain, "MSA (captured seqres templates too)"
    return best_coords_by_chain, "seqres best (captured top-%d templates)" %
        N_TEMPLATES

# Patch DSF
dsf_predict.build_template_init_coords = build_template_init_coords_patched

# ------------------------------
# Timing wrappers
# ------------------------------
TIMERS = {"template": 0.0, "pfsc": 0.0, "pfsc_pure": 0.0}
HITS = {"template": 0}

if not getattr(dsf_predict, "_TIMING_PATCHED", False):
    _orig_bt = dsf_predict.build_template_init_coords
    def _timed_bt(*args, **kwargs):
        t0 = time.perf_counter()
        out, msg = _orig_bt(*args, **kwargs)
        TIMERS["template"] += time.perf_counter() - t0
        if out and any(np.isfinite(v).any() for v in out.values()):
            HITS["template"] += 1
        return out, msg
    dsf_predict.build_template_init_coords = _timed_bt

    _orig_pfsc = dsf_predict.dsf_to_coords_pfsc
    def _timed_pfsc(*args, **kwargs):
        t0 = time.perf_counter()
        out = _orig_pfsc(*args, **kwargs)
        TIMERS["pfsc"] += time.perf_counter() - t0
```

```
        return out
    dsf_predict.dsf_to_coords_pfsc = _timed_pfsc

    dsf_predict._TIMING_PATCHED = True

# ------------------------------
# Run PFSC with templates enabled (main run)
# ------------------------------
test_df = pd.read_csv(COMP_DIR / "test_sequences.csv")
print("\nLoaded test_sequences.csv:", test_df.shape)

t_pred0 = time.perf_counter()
sub_df = dsf_predict.predict_submission(
    test_df,
    dataset_root=str(COMP_DIR),
    base_seed=0,
    chain_mode="separate",
    engine="pfsc",
    pfsc_preset="medium",
    use_templates=True,
    verbose=False,
)
t_pred = time.perf_counter() - t_pred0

# Optional: run template-free PFSC for model_5
sub_df_pure = None
if PURE_PFSC_MODEL5:
    print("\nRunning template-free PFSC for model_5 (use_templates=False)...")
    t0 = time.perf_counter()
    sub_df_pure = dsf_predict.predict_submission(
        test_df,
        dataset_root=str(COMP_DIR),
        base_seed=PURE_PFSC_SEED,
        chain_mode="separate",
        engine="pfsc",
        pfsc_preset="medium",
        use_templates=False,
        verbose=False,
    )
    TIMERS["pfsc_pure"] = time.perf_counter() - t0

# ------------------------------
# Inject templates into models 1..4 as hybrid (template + PFSC fill aligned)
# Leave model_5 as PFSC (optionally pure template-free)
# ------------------------------
sub_df = sub_df.copy()
sub_df["__tid__"] = sub_df["ID"].astype(str).str.split("_").str[0]

# If we ran pure PFSC, replace model_5 coords with pure-run model_1 (any one is fine)
if sub_df_pure is not None:
    sub_df_pure = sub_df_pure[["ID", "x_1", "y_1", "z_1"]].copy()
    sub_df = sub_df.merge(sub_df_pure, on="ID", how="left", suffixes=("", "_pure"))
    # fill only if present
    for ax in ("x","y","z"):
```

```
        sub_df[f"{ax}_5"] = np.where(
            np.isfinite(sub_df[f"{ax}_1_pure"].to_numpy(np.float64)),
            sub_df[f"{ax}_1_pure"].to_numpy(np.float64),
            sub_df[f"{ax}_5"].to_numpy(np.float64),
        )
    sub_df.drop(columns=["x_1_pure","y_1_pure","z_1_pure"], inplace=True)

n_points_aligned = 0
n_points_overridden = 0

def _apply_hybrid_for_target(df_t: pd.DataFrame, tpl_xyz: np.ndarray, model_idx:
    int):
    """
    df_t: rows for one target, sorted by resid
    tpl_xyz: (L,3) with NaNs for missing
    model_idx: 1..4
    """
    global n_points_aligned, n_points_overridden

    cols = [f"x_{model_idx}", f"y_{model_idx}", f"z_{model_idx}"]
    pf = df_t[cols].to_numpy(np.float32, copy=True)
    idx = df_t["resid"].to_numpy(np.int64) - 1
    idx = np.clip(idx, 0, tpl_xyz.shape[0]-1)

    tpl_rows = tpl_xyz[idx].astype(np.float32, copy=False)

    ok_tpl = np.isfinite(tpl_rows).all(axis=1)
    if not np.any(ok_tpl):
        return # nothing to do

    # Align PFSC coords into template frame using overlapping points
    P = pf[ok_tpl].astype(np.float64, copy=False)
    Q = tpl_rows[ok_tpl].astype(np.float64, copy=False)

    if P.shape[0] >= 5:
        R, t = _kabsch_align(P, Q)
        pf_aligned = (pf.astype(np.float64) @ R) + t
        n_points_aligned += int(P.shape[0])
    else:
        pf_aligned = pf.astype(np.float64)

    hybrid = pf_aligned.copy()
    hybrid[ok_tpl] = tpl_rows[ok_tpl].astype(np.float64)

    # write back
    df_t.loc[df_t.index, cols[0]] = hybrid[:,0]
    df_t.loc[df_t.index, cols[1]] = hybrid[:,1]
    df_t.loc[df_t.index, cols[2]] = hybrid[:,2]
    n_points_overridden += int(ok_tpl.sum())

# Process per target
out_frames = []
missing_capture = 0

```

```
for tid, df_t in sub_df.groupby("__tid__", sort=False):
    cap = TEMPLATE_CAPTURE.get(str(tid))
    if not cap or not cap.get("templates"):
        missing_capture += 1
        out_frames.append(df_t)
        continue

    df_t = df_t.sort_values("resid").copy()

    templates = cap["templates"]
    # Apply each template to its model slot (1..min(4, len(templates)))
    for i in range(min(N_TEMPLATES, len(templates))):
        tpl_xyz = templates[i]
        _apply_hybrid_for_target(df_t, tpl_xyz, model_idx=i+1)

    out_frames.append(df_t)

sub_df = pd.concat(out_frames, axis=0, ignore_index=True)
sub_df.drop(columns="__tid__", inplace=True)

# ------------------------------
# Translation-only fix to avoid -999.999 clipping (TM-score is translation invariant)
# ------------------------------
coord_cols = [f"{a}_{k}" for k in range(1, 6) for a in ("x", "y", "z")]
lower = -999.999

sub_df["__tid__"] = sub_df["ID"].astype(str).str.split("_").str[0]
for axis in ("x", "y", "z"):
    cols = [f"{axis}_{k}" for k in range(1, 6)]
    row_min = sub_df[cols].min(axis=1)
    tgt_min = row_min.groupby(sub_df["__tid__"]).transform("min")
    shift = np.where(tgt_min < lower, (lower - tgt_min) + 1e-3, 0.0)
    for c in cols:
        sub_df[c] = sub_df[c] + shift
sub_df.drop(columns="__tid__", inplace=True)

# ------------------------------
# Write submission
# ------------------------------
out_path = Path("/kaggle/working/submission.csv")
t0 = time.perf_counter()
sub_df.to_csv(out_path, index=False)
t_write = time.perf_counter() - t0

cmin = float(sub_df[coord_cols].min().min())
cmax = float(sub_df[coord_cols].max().max())

print("\nVALIDATION PASSED [OK]")
print("Wrote:", out_path)
print("Rows/cols:", sub_df.shape)
print("Coord range:", (cmin, cmax))

print("\nTIMING")
```

```
print(f"template search total: {TIMERS['template']:.1f} sec | template hits:
    {HITS['template']}/{test_df.shape[0]} targets")
print(f"PFSC (templated): {TIMERS['pfsc']:.1f} sec")
if PURE_PFSC_MODEL5:
    print(f"PFSC (template-free): {TIMERS['pfsc_pure']:.1f} sec")
print(f"write CSV: {t_write:.1f} sec")
print(f"TOTAL main predict: {t_pred:.1f} sec")

print("\nHYBRID STATS")
print("Targets missing capture:", missing_capture)
print("Aligned points used (Kabsch):", n_points_aligned)
print("Template points overridden:", n_points_overridden)

print("\nTOP TEMPLATES PER TARGET (first 10 targets shown)")
shown = 0
for tid in sorted(TEMPLATE_CAPTURE.keys()):
    cap = TEMPLATE_CAPTURE[tid]
    print(f"{tid}:")
    for i, (m, cov) in enumerate(zip(cap.get("meta", []), cap.get("cov", []))):
        print(f" model_{i+1}: cov_global={cov:.2f} | {m}")
    shown += 1
    if shown >= 10:
        break

print("\nPreview:")
print(sub_df.head(3))
```